

FORMATION DEVOPS

CI/CD : livrer plus vite, avec confiance

Les bénéfices business de l'agilité technique — illustré avec GitHub Actions

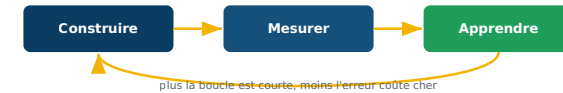
Formateur : Haithem Mihoubi

Comment lire cette présentation

On avance **pas à pas** : pour chaque idée — le problème, la solution, le **bénéfice métier**, puis un exemple.

- 1) Pourquoi le CI/CD ? (le problème du métier)
- 2) Ce que CI/CD apporte : qualité + **agilité business**
- 3) GitHub Actions, composant par composant

Fil rouge : l'application QuickBite, du commit à la prod



Pourquoi le CI/CD ?

Le problème côté métier

Livrer du logiciel, c'est risqué

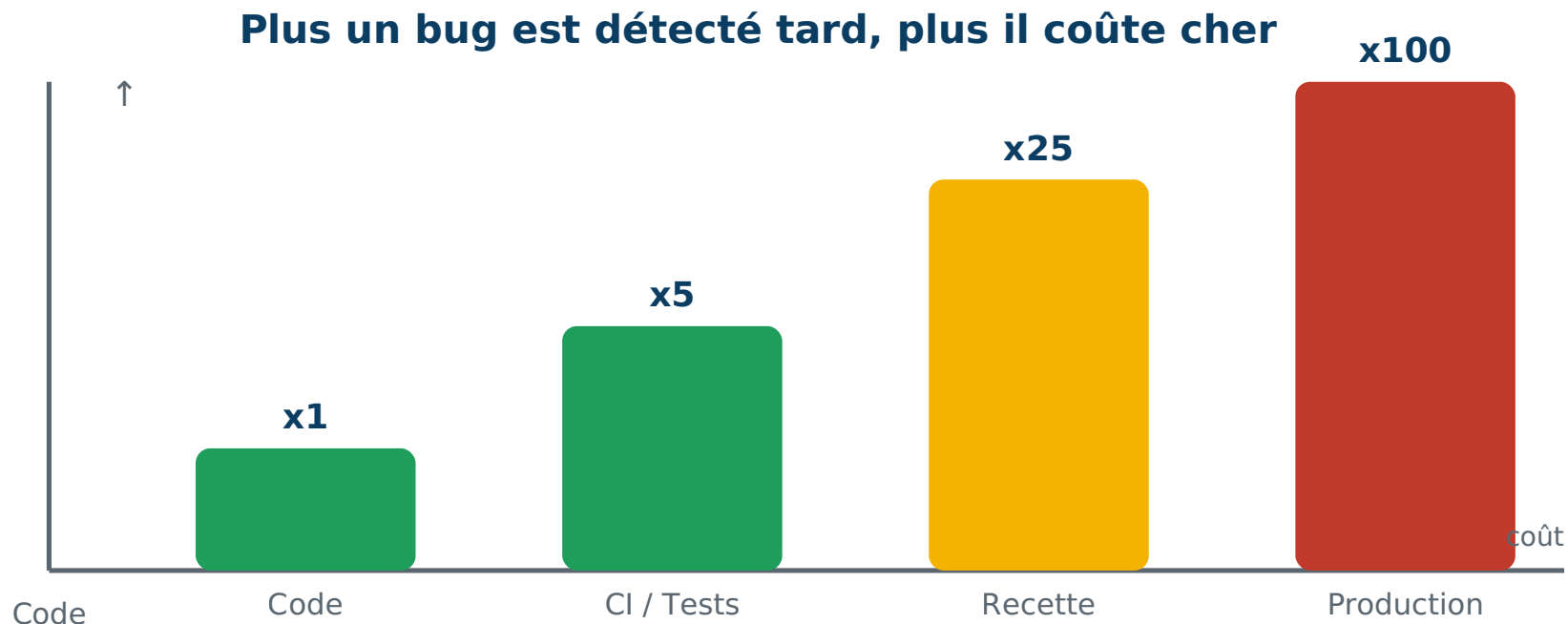
Sans automatisation, chacun code dans son coin et **l'intégration arrive à la fin** : c'est là que tout casse.

- « Ça marche chez moi » → mais pas en production
- Fusions géantes = conflits et bugs en cascade
- Mises en prod rares, manuelles et **stressantes**

Conséquence métier : on livre lentement et on perd des clients

Le coût d'un bug grandit avec le temps

Un bug attrapé **au commit** coûte presque rien ; le même bug **en production** peut coûter 100x. Le CI/CD déplace la détection le plus tôt possible — « shift left ».

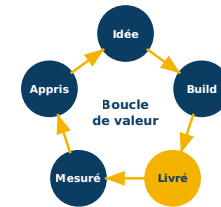


Ce que veut le métier

Le métier ne veut pas « des pipelines ». Il veut **livrer de la valeur vite et sans casser**.

- Mettre une fonctionnalité entre les mains des clients **rapidement**
- Réagir à un concurrent ou à un bug en heures, pas en semaines
- Tenir la promesse de l'**agilité** : apprendre et corriger en boucle courte

CI/CD = le moteur technique qui rend l'agilité réelle



CI/CD raccourcit le tour de boucle → on apprend et on gagne plus vite

CI/CD & ses bénéfices

De la qualité technique à la valeur business

CI — Intégration continue

À chaque **commit**, un pipeline automatique récupère le code, le teste et le construit.

- On intègre **en petits lots, plusieurs fois par jour**
- Lint + tests automatisés à chaque changement
- Règle d'or : **on ne fusionne jamais un pipeline rouge**

Résultat : les bugs sont bloqués avant d'atteindre les autres



CD — Livraison / Déploiement continu

Le code qui passe la CI est **toujours déployable** ; le CD pousse ensuite en production de façon automatique.

- **Livraison continue** : prêt à déployer à tout moment (1 clic)
- **Déploiement continu** : la prod se met à jour toute seule
- Rollback automatique si quelque chose tourne mal

La mise en prod devient une routine, pas un événement

À chaque push : tout est automatisé



✗ tests rouges → fusion bloquée

Bénéfices techniques

Le pipeline est un **filet de sécurité** qui tourne pour vous, jour et nuit.

Qualité constante

tests + lint + scan sécu à chaque commit

Feedback rapide

on sait en minutes si on a cassé quelque chose

Moins de bugs en prod

ils sont attrapés tôt, au plus bas coût

Process reproductible

fini le « ça marche chez moi »



Bénéfices business & agilité

Ces gains techniques se traduisent directement en **valeur pour l'entreprise.**

Time-to-market réduit

la valeur arrive aux clients en jours, pas en mois

Boucle d'apprentissage courte

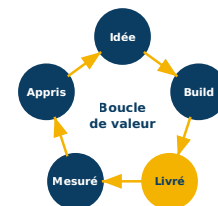
tester une idée, mesurer, ajuster — vite

Confiance & moins de stress

des équipes qui livrent sereinement

Coûts maîtrisés

moins d'incidents, moins de travail refait



CI/CD raccourcit le tour de boucle → on apprend et on gagne plus vite

Mesurer l'impact : les 4 métriques DORA

On ne pilote bien que ce qu'on mesure. Les métriques **DORA** relient la performance technique à la performance business : les organisations « élite » déploient plus souvent ET sont plus stables.

4 métriques DORA : mesurer l'impact business du CI/CD

Fréquence de déploiement

à quelle vitesse on livre



Délai de livraison (Lead Time)

commit → production



Taux d'échec de changement

% de déploiements ratés



MTTR

temps de rétablissement



Exemple chiffré : avant / après

Le même produit, la même équipe — avant et après l'adoption du CI/CD. Livrer **plus souvent** et **plus sûrement** ne sont pas opposés : ils vont ensemble.

Effet concret sur une équipe produit

Indicateur	Avant CI/CD	Avec CI/CD
Fréquence de livraison	1× / trimestre	10× / jour
Délai commit → prod	3 semaines	15 minutes
Bugs détectés en prod	fréquents	rare (bloqués en CI)
Retour arrière (rollback)	heures, manuel	1 clic, auto
Stress des mises en prod	élevé	routine

PARTIE 3

GitHub Actions

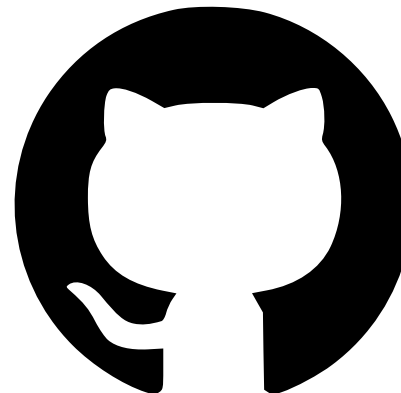
Le CI/CD intégré à votre dépôt

| Pourquoi GitHub Actions ?

Le CI/CD **natif dans GitHub** : pas de serveur séparé à installer, tout vit avec le code.

- Configuré en **YAML**, versionné avec le projet
- Runners hébergés (Linux/Windows/macOS) — gratuit pour le public
- **Marketplace** : des milliers d'actions réutilisables

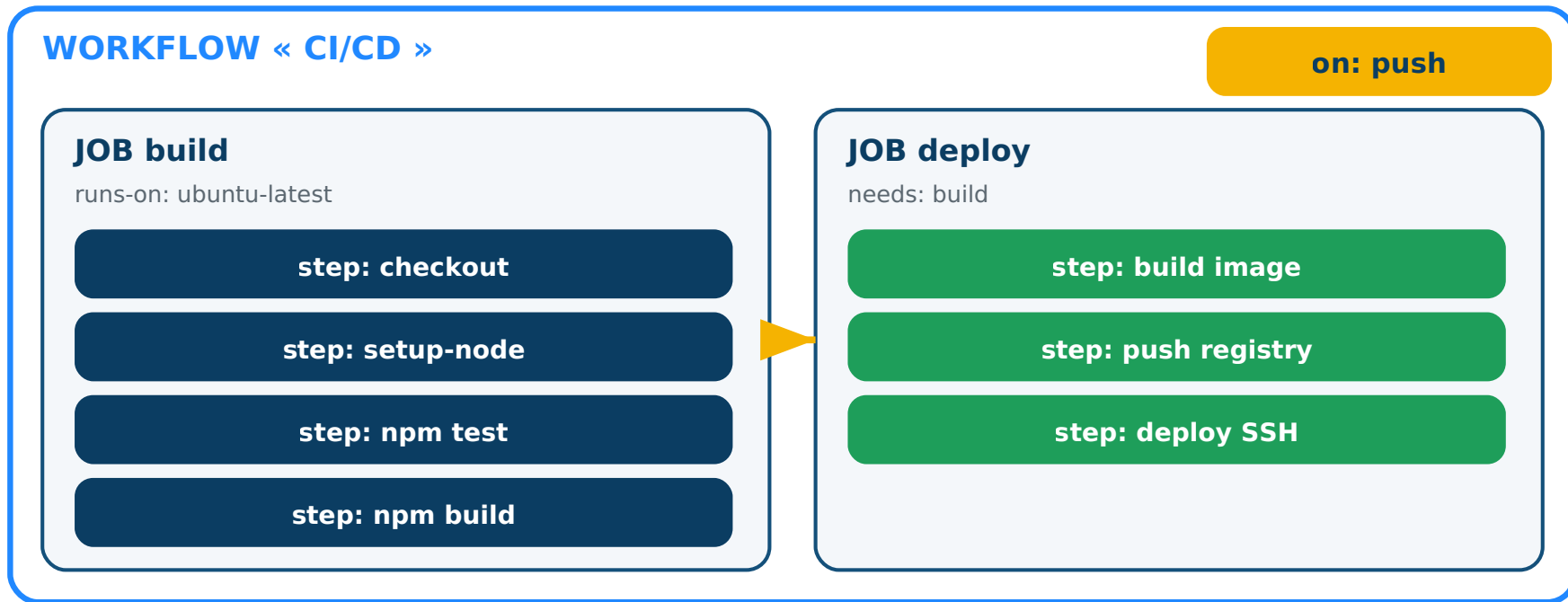
Déclenché par un push, une PR, un planning, un bouton...



Anatomie : Workflow ► Job ► Step

Un **Workflow** contient des **Jobs** (qui tournent en parallèle par défaut), chaque Job contient des **Steps** exécutés dans l'ordre. `needs` : permet d'enchaîner les jobs.

Workflow = 1 fichier `.yaml` dans `.github/workflows/`



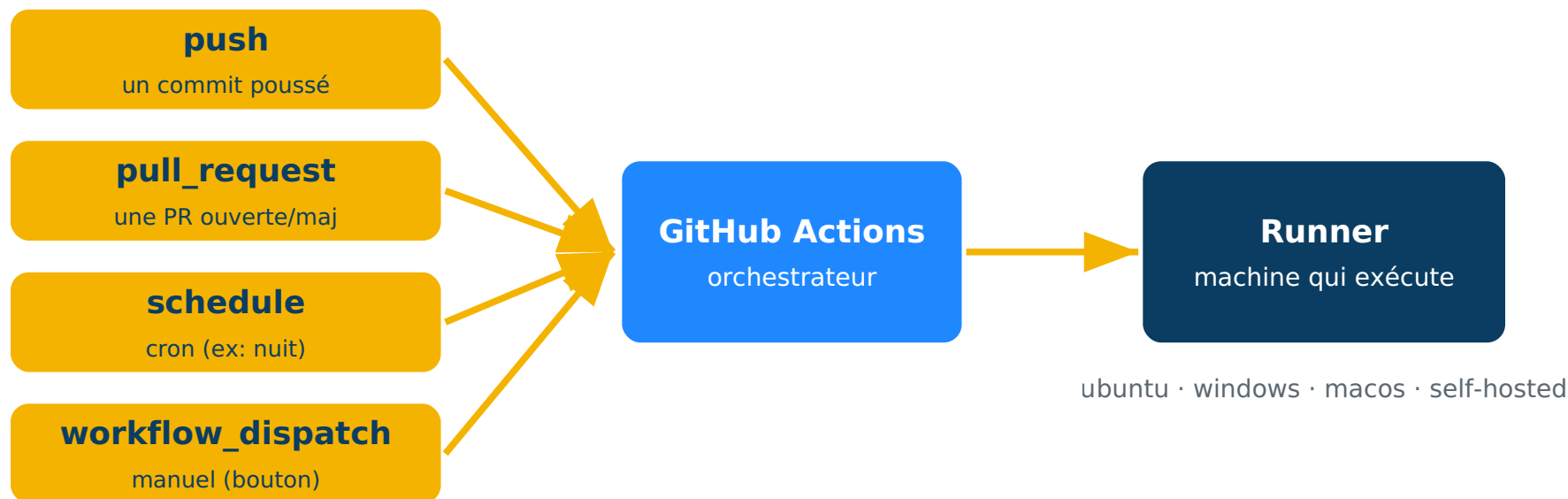
Les composants à connaître

Huit briques suffisent à tout comprendre. On les retrouve dans chaque workflow.

Les événements déclencheurs

Le mot-clé **on**: définit **quand** le workflow se lance. `push` et `pull_request` couvrent l'essentiel du CI ; `workflow_dispatch` ajoute un déclenchement manuel.

Un événement déclenche un workflow sur un runner



Un workflow complet, expliqué

`.github/workflows/ci.yml`

```
name: CI
on:
  push:
    branches: [ main ]
  pull_request:
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm test
      - run: npm run build
```

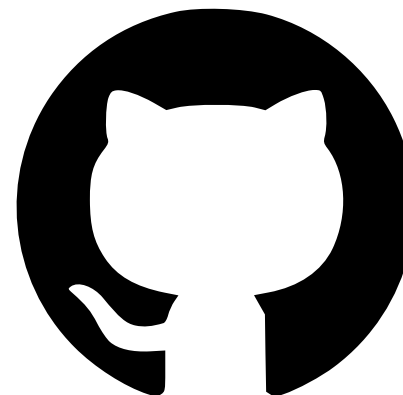
- **on** : déclenché par push sur main et par toute PR
 - **jobs.build** : 1 job, sur un runner Ubuntu
 - **uses** : on réutilise des actions du Marketplace
 - **run** : on exécute nos commandes shell
- Si un step échoue → le job est rouge → la PR est bloquée

Secrets & sécurité

Jamais de mot de passe en clair dans le YAML : on utilise des **Secrets** chiffrés.

- Stockés dans **Settings ► Secrets**, masqués dans les logs
- Lus via `${ secrets.SSH_KEY }`
- Permissions minimales du GITHUB_TOKEN (least privilege)

Un scan de sécurité dans le pipeline = bénéfice sécu en plus



Cas pratique

QuickBite : du commit à la production

QuickBite : le pipeline en action

Un développeur pousse une fonctionnalité. Voici ce qui se passe **sans intervention humaine**.

- 1. Push → GitHub Actions démarre le workflow
- 2. Tests + lint + build →  vert
- 3. Image Docker construite et poussée au registry
- 4. Déploiement automatique → clients servis en **minutes**

À chaque push : tout est automatisé



X tests rouges → fusion bloquée

Synthèse : CI/CD = agilité concrète

La technique au service du métier : **livrer de la valeur, vite et en confiance.**

Agilité

boucle idée → prod → feedback raccourcie

Qualité

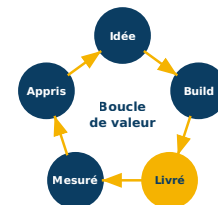
le pipeline protège chaque livraison

Valeur business

time-to-market et satisfaction client

GitHub Actions

le moteur, intégré et simple à adopter



CI/CD raccourcit le tour de boucle → on apprend et on gagne plus vite

Merci !

Questions / Réponses — Haithem Mihoubi